

CS 170 Midterm 1

Write in the following boxes clearly and then double check.

Name :

SID :

Exam Room :

☐ EVAN 10 ☐ GPBB 100 ☐ HFAXA 1
☐ VLSB 2050 ☐ VLSB 2040
 Other (Specify):

Name of student to your left :

Name of student to your right :

- The exam will last 110 minutes.
- The exam has 11 questions with a total of 120 points. You may be eligible to receive partial credit for your proof even if your algorithm is only partially correct or inefficient.
- Only your writings inside the answer boxes will be graded. **Anything outside the boxes will not be graded.** The last page is provided to you as a blank scratch page.
- Answer all questions. Read them carefully first. Not all parts of a problem are weighted equally.
- Be precise and concise.
- The problems may **not** necessarily follow the order of increasing difficulty.
- The boxes assigned to each problem are by no means an indication of the problem's difficulty.
- Unless the problem states otherwise, you should assume constant time arithmetic on real numbers. Unless the problem states otherwise, you should assume that graphs are simple.
- If you use any algorithm from lecture and textbook as a blackbox, you can rely on the correctness and time/space complexity of the quoted algorithm. If you modify an algorithm from textbook or lecture, you must explain the modifications precisely and clearly, and if asked for a proof of correctness, give one from scratch or give a modified version of the textbook proof of correctness.
- Assume the subparts of each question are **independent** unless otherwise stated.
- Please write your SID on the top of each page.
- Good luck!

1 Asymptotic Analysis (4 points)

For each pair of functions f and g , specify whether $f = O(g)$, $g = O(f)$, or both.

f	g	$f = O(g)$	$g = O(f)$	Both
$n(\log n)^{2023}$	$n^2/(\log n)^{2023}$	☐	☐	☐
n^n	2^{100n}	☐	☐	☐
$n^2 + (\log n)^3$	$n + (\log n)^6$	☐	☐	☐

Solution:

f	g	$f = O(g)$	$g = O(f)$	Both
$n(\log n)^{2023}$	$n^2/(\log n)^{2023}$	☒	☐	☐
n^n	2^{100n}	☐	☒	☐
$n^2 + (\log n)^3$	$n + (\log n)^6$	☐	☒	☐

2 Runtime Analysis (6 points)

Consider the following piece of code:

```

Function what( $n$ ) {
  If ( $n < 2$ ):
    return;
   $k \leftarrow n$ 
  while  $k > 1$  do {
    for  $j = 1, 2, 3, \dots, n^2$  {
      print BLAH
    }
     $k \leftarrow k/16$ 
  }
  what( $n/2$ )
}

```

Let $T(n)$ denote the runtime of the `what( $n$ )`.

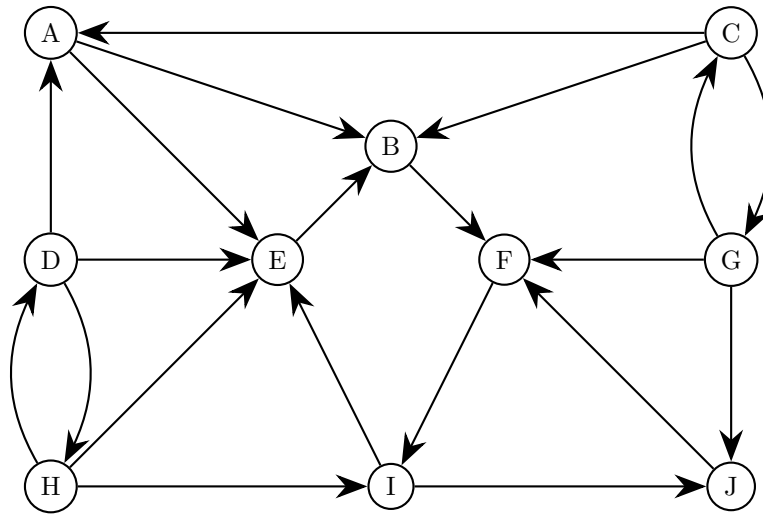
- Write the recurrence relation for $T(n)$

Solution: $T(n) = T(n/2) + O(n^2 \log n)$

- Solve the recurrence relation for $T(n)$. Write down the tightest bound $O()$ possible, proof is not needed.

Solution: $O(n^2 \log(n))$

3 Connectivity in Graphs (8 points)



1. Perform DFS in the graph starting at node A and write down the pre and post numbers. Break ties alphabetically.

Vertex v	$pre[v]$	$post[v]$
A		
B		
C		
D		
E		
F		
G		
H		
I		
J		

Solution:

Vertex v	$pre[v]$	$post[v]$
A	1	12
B	2	11
C	13	16
D	17	20
E	5	6
F	3	10
G	14	15
H	18	19
I	4	9
J	7	8

2. List all cross edges if any.

Edge	Fill if cross edge	Edge	Fill if cross edge
$A \rightarrow B$	☐	$F \rightarrow I$	☐
$A \rightarrow E$	☐	$G \rightarrow C$	☐
$B \rightarrow F$	☐	$G \rightarrow F$	☐
$C \rightarrow A$	☐	$G \rightarrow J$	☐
$C \rightarrow B$	☐	$H \rightarrow D$	☐
$C \rightarrow G$	☐	$H \rightarrow E$	☐
$D \rightarrow A$	☐	$H \rightarrow I$	☐
$D \rightarrow E$	☐	$I \rightarrow E$	☐
$D \rightarrow H$	☐	$I \rightarrow J$	☐
$E \rightarrow B$	☐	$J \rightarrow F$	☐

Solution: $CA, CB, DA, DE, GF, GJ, HE$, and HI are the cross edges.

3. In this class, we have learnt an algorithm to compute the strongly connected components (SCCs) of a directed graph. Write down the strongly connected components (SCCs) in the above graph in the same order in which the algorithm outputs them. If there are multiple possible orderings, write the lexicographically smallest one.

SCC 1:		SCC 2:	
SCC 3:		SCC 4:	
SCC 5:		SCC 6:	
SCC 7:		SCC 8:	

Solution: There are two possible orderings of the 4 SCCs.

$\{B, E, F, I, J\} \{A\} \{C, G\} \{D, H\}$

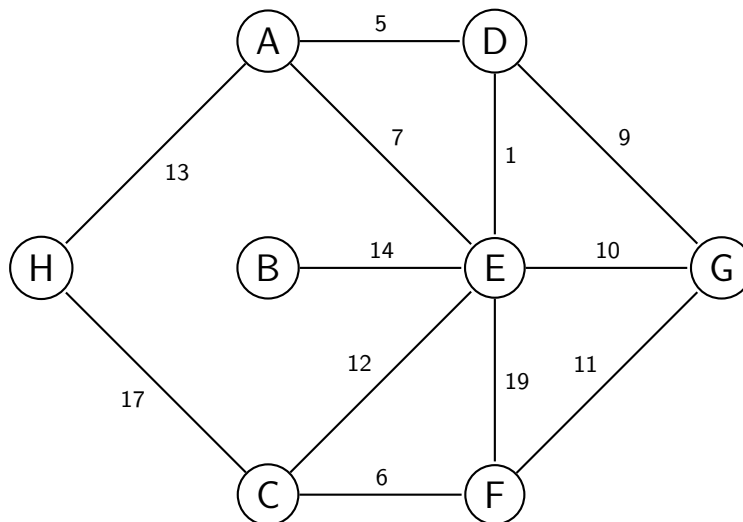
and

$\{B, E, F, I, J\} \{A\} \{D, H\} \{C, G\}$

we accepted either ordering for full credit.

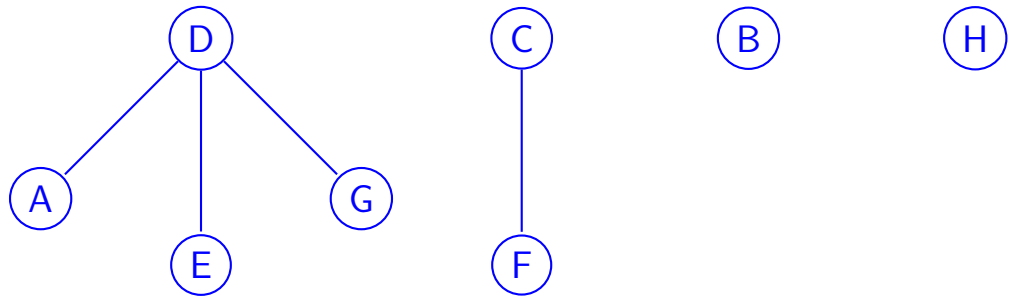
4 MST Execution (8 points)

1. In the graph below, what would be the fourth and fifth edges added by Kruskal's algorithm?

Fourth edge: Fifth edge:

Solution: Fourth edge: *DG* Fifth edge: *FG*

2. Draw the union-find trees (no path compression) after the inclusion of the fourth edge.



Solution:

3. In the same graph, what would be the fourth and fifth edges added by Prim's algorithm started at A ?

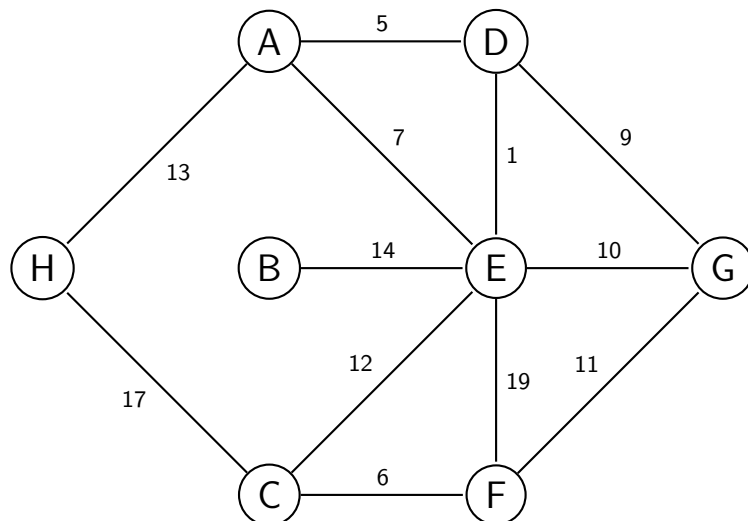
Fourth edge:

Fifth edge:

Solution: Fourth edge: FG Fifth edge: CF

4. Suppose we run Prim's algorithm and Dijkstra's algorithm, both starting from A . Consider the step at which the two algorithms delete different nodes from the priority queue for the first time. In this step, Prim's algorithm deletes from priority queue, while Dijkstra's algorithm deletes .

We have reproduced the graph again for your convenience.



Note that in both algorithms, the first vertex to be deleted from the priority queue is the vertex A .

Solution: In the third step of each algorithm, Prim's will delete node G while Dijkstra's will delete node H .

5 Polynomial multiplication via FFT (6 points)

In this question, we will run through a polynomial multiplication example via the Fourier transform. Consider two polynomials:

$$P(x) = 1 + x \quad Q(x) = 1 + 2x^2$$

Suppose we were to compute the product of these two polynomials $P(x)$ and $Q(x)$, using the Fourier transform.

1. What is the smallest n for which we can use the n^{th} root of unity?

Solution: 4

2. For the above value of n , list the n^{th} roots of unity.

Solution: $[1, i, -1, -i]$

3. What is the Fourier transform associated with the polynomial P ?

Solution: $[2, 1 + i, 0, 1 - i]$

4. What is the Fourier transform associated with the polynomial Q ?

Solution: $[3, -1, 3, -1]$

5. What is the Fourier transform associated with the product of polynomials P and Q ? (No need to simplify the expressions completely)

Solution: $[2 \cdot 3, (1 + i) \cdot (-1), 0 \cdot 3, (1 - i) \cdot (-1)] = [6, -(1 + i), 0, i - 1]$.

6 Short Answers (23 points)

1. A document consists of symbols A, B, C, D, E with frequencies $1, 3, 3^2, 3^3, 3^4$ respectively.

Suppose we run Huffman encoding on these letters, the letter C will be encoded using bits.

Solution: 3

2. Let $Select(S, k)$ denote the randomized Select algorithm that finds the k^{th} smallest number in a set S .

Consider the execution of $Select(a[1, \dots, n], 1)$ on an array of distinct integers $a[1, \dots, n]$.

What is the probability that this algorithm needs $n - 1$ recursive calls before returning the 1^{st} -smallest element?

(Here $n - 1$ recursive calls do not include the outer call $Select(a[1, \dots, n], 1)$)

Solution: $\frac{1}{n!}$

3. Strassen's matrix multiplication algorithm uses fewer multiplications than the naive algorithm for matrix multiplication. However, it uses more additions than the naive algorithm.

☐ True ☐ False

Solution: The intended interpretation of the question was:

Strassen's matrix multiplication algorithm uses fewer multiplications (**between numbers** than the naive algorithm for matrix multiplication. However it uses more additions (**and subtractions between numbers**) than the naive algorithm.

The above assertion would be false, since Strassen's algorithm uses at most $O(n^{2.8})$ additions and multiplications, while naive algorithm uses $\Theta(n^3)$ additions and multiplications. However, since the statement could potentially have other interpretations, the question was dropped.

4. In a directed acyclic graph (DAG), if $pre[u] < pre[v] < post[v] < post[u]$ then edge is necessarily absent.

Solution: $v \rightarrow u$

5. A DAG on n vertices can have at least SCCs and at most SCCs.

Solution: n and n

6. Fast Fourier transform (FFT) algorithm on vector $[1, 2, 3, 4, 1, 2, 3, 4]$ makes two recursive calls to FFT on 4-dimensional vectors. Specifically, the calls are for FFT on vectors

and .

Solution: $[1, 3, 1, 3]$ and $[2, 4, 2, 4]$.

7. If Fourier transform of $[a, b, c, d]$ is $[\alpha_0, \alpha_1, \alpha_2, \alpha_3]$, then Fourier transform of $[a, 0, b, 0, c, 0, d, 0]$ is .

Solution: $[\alpha_0, \alpha_1, \alpha_2, \alpha_3, \alpha_0, \alpha_1, \alpha_2, \alpha_3]$

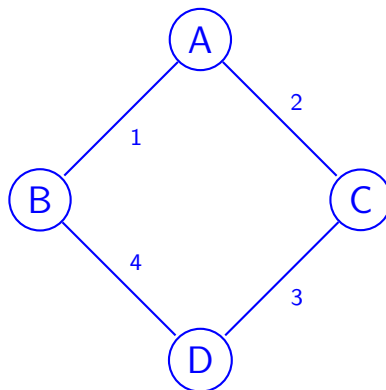
8. On a graph with n vertices and m edges, Dijkstra's algorithm executes many DeleteMin operations and DecreaseKey operations on its priority queue.

Solution: n deletemin operations and at most m decreaseKey operations. $O(m)$ and $\leq m$ were also accepted answers for number of decreaseKey operations.

9. If the edge-lengths of a graph are distinct then the shortest path between any two nodes is unique.

☐ True ☐ False

Solution: False. Consider the following graph:



Although all the edge-lengths in the graph are unique, both of the paths (A, B, D) and (A, C, D) have the same (shortest) length.

10. In a graph with positive and negative weights, if we cube all the edge-weights in a graph, every MST continues to be an MST.

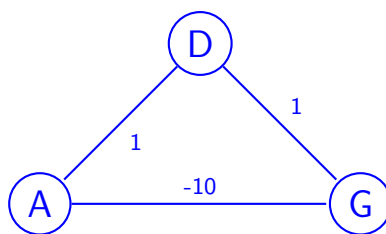
☐ True ☐ False

Solution: True. Cubing all the edge-weights in the graph preserves the relative ordering of edges.

11. In a graph with positive and negative weights, if we square all the edge-weights in a graph, every MST continues to be an MST.

☐ True ☐ False

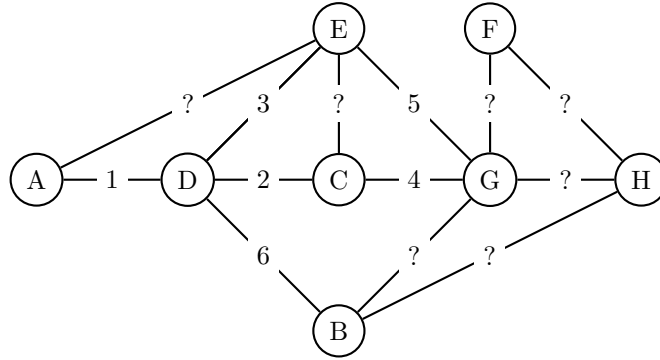
Solution: False. Consider the triangle graph G below:



The two MSTs in G are $\{AD, AG\}$ and $\{DG, AG\}$, while the sole MST in G with all edge-weights squared is $\{AD, DG\}$.

7 MST with some edge-weights unknown (10 points)

In this graph, some of the edge weights are known, while the rest are unknown.



$$\text{cost}(A, D) = 1, \text{cost}(C, D) = 2, \text{cost}(D, E) = 3, \text{cost}(C, G) = 4, \text{cost}(E, G) = 5, \text{cost}(B, D) = 6$$

There are two edges that **must** necessarily belong to a minimum spanning tree, regardless of what the unknown edge weights turn out to be. Justify each of your edges briefly (a sentence or less is enough).

1. Edge necessarily belongs to an MST because

Solution: Edge AD because it is the shortest edge across the cut $\{D\}, \{A, B, C, E, F, G, H\}$.

2. Edge necessarily belongs to an MST because

Solution: Edge CG because it is the shrotest edge across the cut $\{A, C, D, E\}$ and $\{B, F, G, H\}$.

Moreover, there is one edge that is necessarily not part of any MST:

1. Edge is necessarily NOT part of any MST because

Solution: Edge EG since it is the longest edge on the cycle $E - D - C - G - E$.

Solution: The path with the longest number of hops from S to any vertex in the graph, has 11 hops. This is the path of length 11 from S to bottom-left corner, that winds around.

Therefore, after 11 iterations of the outer loop, for every path P from S to any vertex in the graph, the edges of P are updated in order. In Bellman-Ford algorithm, the distance value to a vertex v converges to the correct value, as soon as all edges along shortest path from S to v are updated in order.

9 Vacation home (15 points)

You are managing a vacation home for D days. You have received bids from n customers for renting the vacation home. Customer i has submitted a bid of the form:

$(startDate_i, endDate_i, rentPerDay_i)$

which means that Customer i wants to rent the vacation home from date $startDate_i$ to date $endDate_i$, and will pay $rentPerDay_i$ for renting it per day.

Devise an algorithm to determine which subset of customers to rent to, in order to maximize the total profit.

Here is the formal description of the problem.

Formal description:

Input:

- There are n customer bids given by $(startDate_i, endDate_i, rentPerDay_i)$ for $i = 1, \dots, n$.
- All start dates and end dates are in the range $\{1, \dots, D\}$ and $rentPerDay_i$ is a positive number. Moreover, $startDate_i < endDate_i$ for each i .

Output: The subset $S \subseteq \{1, \dots, n\}$ of customers to whom we rent the vacation home, so as to maximize the total rent collected, while satisfying the following constraints:

- On any given day, the vacation home can be rented to at most one customer.
- Customer i will either rent the home for the entire period between $startDate_i$ to $endDate_i$ or will not rent at all.
- The $endDate$ of a customer renting the home can be the same as the $startDate$ of the next customer renting it.

-
1. Give a succinct and precise description of the algorithm:

Answer box on next page.

Solution: Create a graph G as follows:

- There are D nodes corresponding to the D days.
- For each customer i , add a directed edge from $startDate_i \rightarrow endDate_i$ with edge weight $-rentPerDay_i \cdot (endDate_i - startDate_i)$.
- For each day $j \in \{1, \dots, D-1\}$, add a directed edge from $j \rightarrow (j+1)$ with edge weight 0.

Run a shortest-path algorithm (e.g. Bellman-Ford or Shortest path on DAG) on the graph G to find the shortest path from node 1 to node D .

Alternate Solution:

Create a graph of n nodes, one for each bid:

- Add a directed edge from u to v if $endDate_u \leq startDate_v$, with weight equal to the total profit gained from bid v : $(endDate_v - startDate_v) \cdot rentPerDay_v$. Doing this will result in a DAG.
- Finally, add a dummy node with an edge to each source node (in the DAG), with weight again equal to the total profit gained from the source node's bid.

- Negate edge weights and run a shortest path algorithm from the dummy node. Output the bids corresponding to nodes that the longest path goes through.

Run a shortest-path algorithm (e.g. Bellman-Ford $O(n^3)$ or Shortest path on DAG $O(n^2)$) on the graph G to find the shortest path from node 1 to node D .

2. What is the runtime of your algorithm?

Solution: $O(D(n + D))$ if using Bellman-Ford or $O(D + n)$ if using the DAG shortest path algorithm.

Alternate: $O(n^3)$ if using Bellman-Ford or $O(n^2)$ if using the DAG shortest path algorithm.

10 Airline Routes (15 points)

Suppose you are running a travel agency that helps customers book air-tickets. The preference of customers depends on number of hops, flight duration and cost. Here you will design an algorithm to find the most preferred route for customers.

The formal description of the problem is given below.

Formal description:

Input:

- A directed graph $G = (\{1, \dots, n\}, E)$ whose vertices are cities, and each directed edge $i \rightarrow j$ is a flight from city i to city j .
- For each directed edge $i \rightarrow j$, there are two positive integer weights:
 - $duration[i, j]$: the duration of the flight from $i \rightarrow j$, always an integer in $\{1, \dots, B\}$.
 - $cost[i, j]$: the price of a ticket on flight $i \rightarrow j$, always an integer in $\{1, \dots, C\}$.
- A start node s and a destination node t .

Output: Find the most preferred route from s to t where the customer's preferences are as follows:

- Among two paths P_1, P_2 from s to t , the customer always prefers the one with fewer hops. (Number of hops is the number of edges on the path).
- If two paths P_1 and P_2 have the same number of hops, then customer prefers the one with shorter total duration.
- If two paths P_1 and P_2 have the same number of hops and the same total duration, then customer prefers the one with lower total cost.

-
1. Devise an algorithm to find the most preferred path from start node s to destination node t .
Give a succinct and precise description of the algorithm. (Proof of correctness is not necessary).

(Answerbox on the next page)

Solution: Define the edge weight $w[i, j]$ as,

$$w[i, j] = \text{cost}[i, j] + (C + 1) \cdot n \cdot \text{duration}[i, j] + (C + 1) \cdot (B + 1) \cdot n^2$$

and run Dijkstra's algorithm with weights $w[i, j]$ on directed edge $i \rightarrow j$.

Alternate Solution: Run Dijkstra's algorithm with a modified heap that orders each vertex v based on the following priority:

(number of hops to v , total duration along route to v , total cost along route to v).

2. What is the runtime of your algorithm?

Solution: The graph construction takes $O(n + |E|)$ time, including the special edge weight definition. Then, Dijkstra's takes $O((n + |E|) \log n)$ time (assuming we use a binary heap). Thus, the overall runtime is

$$O(n + |E|) + O((n + |E|) \log n) = O((n + |E|) \log n).$$

Runtime for Alternate Solution: All priority queue operations are scaled up by a factor of $O(3) = O(1)$, meaning they remain the same asymptotically. Thus, running Dijkstra's takes $O((n + |E|) \log n)$ time.

11 Bikeable cities (20 points)

(We recommend that you attempt this question after finishing other questions on the exam)

We have a road network $G = (V, E)$ between a set of cities. Each city v is at an altitude $h[v]$.

Two cities $\{u, v\}$ are said to be *bikeable* from one another, if we can bike u to v and back. The crucial constraint is that bikes can't go uphill, i.e., bikes can't go from one city to a neighboring city with a higher altitude.

For each city u , we would like to find some partner city v such that $\{u, v\}$ are *bikeable* from one another.

Note: It was clarified during the exam that bikes actually *can* go uphill; ignore the sentence “The crucial constraint is that bikes can't go uphill...”. Refer to the formal description of the problem below.

Here is the formal description of the problem.

Formal description:

Input:

1. An undirected graph $G = (V, E)$.
2. For each vertex $v \in V$, we are given its altitude $h[v]$. $h(v)$ is a positive integer.

A pair of vertices $\{u, v\}$ are said to be *bikeable from each other* if

- Altitudes of u and v are the same, $h[u] = h[v]$.
- There is some path P from u to v , where all the intermediate vertices have altitude at most $h[u]$ (and $h[v]$).

Output:

1. For every vertex u , find some vertex v such that $\{u, v\}$ are bikeable from each other, if there exists at least one such vertex v .
-

Continued on next page.

1. Succinctly and precisely describe an algorithm for the problem. (For **non-zero credit**, your algorithm should run in time $O((|V| + |E|) \cdot \log |V|)$. Faster run-times mean higher credit.)

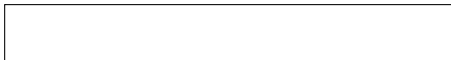
[illegible]

2. What is the runtime of your algorithm?

--

Solution: The algorithm will proceed as follows:

- Sort the vertices in increasing order of altitude. Let $v_1, \dots, v_n$ denote the sorted order.
- Starting with the empty graph, add one vertex at a time in the order $v_1, \dots, v_n$. Formally, let G_i denote the graph after adding $v_1, \dots, v_i$. In the $i + 1^{st}$ step, we add vertex v_{i+1} and all edges between v_{i+1} and $\{v_1, \dots, v_i\}$.



- We maintain the connected components of the graphs $\{G_i\}$ (using the union-find data structure). Moreover, for each connected component in the union find data structure, we keep track of the vertex in that component with the highest altitude.
- On inserting v_{i+1} into a component C_j , if the highest altitude vertex in C_j is u and $h[u] = h[v_{i+1}]$ then output the pair $\{u, v_{i+1}\}$. Else, update the highest altitude vertex C_j to be v_{i+1} .

The runtime of sorting is $|V| \log |V|$. Following this, there are $O(|V| + |E|)$ union and find operations in the data structure. Hence, the total runtime (without path compression) is

$$O(|V| \log |V| + (|V| + |E|) \log(|V|)) = O((|V| + |E|) \log |V|).$$

Note that using union-find with path compression improves the time required for the union and find operations, yielding a runtime of:

$$O(|V| \log |V| + (|V| + |E|) \log^*(|V|)) = O(|V| \log |V| + (|V| + |E|) \log^* |V|).$$

This optimization was not required for full credit.